Codex Grail

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Codex Grail | Interactive Translations Tracker</title>
  <meta name="description" content="Celebrating the lists of JRR. Marks and charts our numerous together" />
  <meta name="author" content="Codex Grail" />
  <link rel="canonical" href="https://kari.grailed.com/" />
  <link rel="icon" href="data:image/svg+xml,%3Csvg%20xmlns=%27http://www.w3.org/2000/svg%27%20viewBox=%270%20,0,100,100%27%3E%3Ctext%20y=%2290%27%20fill=%27%23c9a%27%20font-size=%27100%27%20text-anchor=%27middle%27%20x=%27100%27%20dx=%27-0.5em%27%20dy=%27-0.5em%27%20transform=%27translate(-50,-50)%27%20stroke-width=%27.5%27%20stroke-linejoin=%27round%27%20stroke-linecap=%27round%27%20stroke=%27rgb(0,0,0)%27%20stroke-opacity=%27.0%27%20paint-order=%27stroke%27%20stroke-dasharray=%27.5,.5,.5,
```

```
    .inspiration-photo.show-selected { border-color: var(--primary-gold); }
    .inspiration-photo.big { width: 100%; height: 100%; object-fit: cover;
    .inspiration-photo.add-more { aspect-ratio: 1/1; display: flex; gap: 8px; width: 40px; height: 40px; background: var(--primary-gold); border-radius: 50%; display: none; align-items: center; justify-content: center; color: var(--navy-dark); font-weight: bold; }
    .inspiration-photo.show-selected .photo-check { display: flex; }

    .inspiration-items-preview { display: flex; gap: 10px; overflow-x: auto; padding-top:8.0px;-bottom:8.0px;-left:0.0px;-right:0.0px; }
    .inspiration-item-preview { flex-shrink: 0; text-center; max-width: 60px; }
    .inspiration-item-preview.big { width: 100%; aspect-ratio: 1/1; object-fit: cover; border-radius: 8px; }
    .inspiration-item-preview.big { width: 100%; aspect-ratio: 1/1; object-fit: cover; border-radius: 8px; }
    .inspiration-item-preview.add-more { aspect-ratio: 1/1; display: flex; gap: 8px; width: 40px; height: 40px; background: rgba(0,0,0,0.5); border: 1px solid var(--glass-border); border-radius: 8px; color: white; }
    .inspiration-item-preview .remove-item { display: block; width: 100%; height:5px; padding: 0px; background: rgba(255,0,0,0.8); border: none; border-radius: 0px; color: #FFFFFF; cursor: pointer; font-size: 0.7em; }

    /* Active add mode */
    .inspiration-photo.add-mode { aspect-ratio: 1/1; display: flex; gap: 8px; box-shadow: 0px; transition: var(--transition-smooth); }
    .inside.add-mode { display: flex; gap: 8px; box-shadow: 0px; transition: var(--transition-smooth); }
    .inside.add-mode { background: rgba(255,0,0,0.8); color: white; }
    .inside.add-mode { background: rgba(255,0,0,0.8); color: white; }
    .inside.add-mode { background: rgba(255,0,0,0.8); color: var(--navy-dark); padding: 0px 8px; border-radius: 8px; font-family: inherit; font-size: inherit; width: 100%; }
    .inside.add-mode { background: rgba(255,0,0,0.8); color: var(--navy-dark); }

</Page>

<Page>

<body>
<!-- Loading Overlay -->
<div id="loadingOverlay" class="loading-overlay">
  <noscript>
    <div id="noJsMessage" class="hidden" >
      <h2>JavaScript Required</h2>
      <p>This application requires JavaScript to function properly.</p>
      <p>Please enable JavaScript in your browser settings and refresh the page.</p>
    </div>
  </noscript>
  <div id="loadingMessage">Loading...</div>
</div>

<div id="toastContainer" class="toast-container" aria-live="polite" aligned="right" />

<div id="mainContent" class="hidden">
  <div id="header" />
  <div id="header" />
  <div id="header" />
```

```
    .hacker-image {
      width: 240px; height: 240px;
      margin: 0 auto 20px;
      height: 0 auto 40px;
      background: var(--gradient-gold);
      display: flex; align-items: center; justify-content: center;
      clip-path: circle(50%);
      overflow: hidden;
      position: relative;
      font-weight: 900;

    .hacker-image-bg { width: 100%; height: 100%; object-fit: cover; }
    .hacker-image-fallback { display: flex; flex-shrink: 0; line-height: 1; font-weight: 900; opacity:0.0.0; 0;
    .hacker-subject { color: var(--primary-gold); font-size: 0.85rem; margin-bottom: 10px; }
    .hacker-image-fallback { display: flex; flex-shrink: 0; line-height: 1; }

    .Mission-Critical-Upgrade-Card { var(--bg-secondary);
    .hacker-image { position: relative; aspect: 300px; height: 0 auto; }
    .hacker-image-fallback { display: flex; flex-shrink: 0; line-height: 1; position: relative; height:300px; width: 0px; height: 300px; background: var(--gradient-gold); border-radius: 0px; }
    .hacker-image-bg { display: flex; flex-shrink: 0; line-height: 1; flex-and; padding-right: 0px; position: relative; height:300px; }
    .hacker-image-fallback { display: flex; flex-shrink: 0; line-height: 1; flex-and; padding-right: 0px; padding-left: 0
```

```
            <label>Category</label>
            <select class="form-select" id="editCtg">
              <option value="Technology">Technology</option>
              <option value="Finance">Finance</option>
              <option value="Lifestyle">Lifestyle</option>
              <option value="Health">Health</option>
              <option value="Entertainment">Entertainment</option>
              <option value="Sports">Sports</option>
              <option value="Other">Other</option>
            </select>
          </div>
          <div class="form-group" style="margin-top: 20px;">
            <label>Thumbnail URL (optional, leave blank to keep current image)</label>
            <input id="editThumbnail" type="text" placeholder="https://...">
          </div>
          <div id="editPreviewWrap" style="display:none;">
            
          </div>
          <div style="text-align:right;">
            <button id="editCancel" type="button">Cancel</button>
            <button id="editSave" type="button">Save Changes</button>
          </div>
        </div>
      </div>
      <div id="confirmOverlay" aligned="center" hidden="" aria-hidden="true">
        <div id="confirmBox" role="alertdialog" aligned="center" aria-modal="true">
          <p id="confirmMsg">Are you sure?</p>
          <div aligned="center">
            <button id="confirmCancel">Cancel</button>
            <button id="confirmOk">Delete</button>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>
```


```
/*!
* Chart.js v4.4.1
* https://www.chartjs.org
* (c) 2023-2023 Chart.js Contributors
* Released under the MIT License
*/
var Chart=(t=>{
var e=Object.
var i=Object.
var s=Object.
var n=Object.
var o=Object.
var a=Object.
var r=Object.
var l=Object.
var h=Object.
var c=Object.
var d=Object.
var u=Object.
var p=Object.
var f=Object.
var g=Object.
var m=Object.
var b=Object.
var x=Object.
var y=Object.
var _=Object.
var w=Object.
var k=Object.
var M=Object.
var S=Object.
var P=Object.
var D=Object.
var C=Object.
var A=Object.
var T=Object.
var O=Object.
var R=Object.
var E=Object.
var L=Object.
var I=Object.
var z=Object.
var B=Object.
var F=Object.
var V=Object.
var W=Object.
var N=Object.
var H=Object.
var U=Object.
var j=Object.
var K=Object.
var X=Object.
var Y=Object.
var Q=Object.
var Z=Object.
var J=Object.
var G=Object.
var tt=Object.
var et=Object.
var it=Object.
var st=Object.
var nt=Object.
var ot=Object.
var at=Object.
var rt=Object.
var lt=Object.
var ht=Object.
var ct=Object.
var dt=Object.
var ut=Object.
var pt=Object.
var ft=Object.
var gt=Object.
var mt=Object.
var bt=Object.
var xt=Object.
var yt=Object.
var _t=Object.
var wt=Object.
var kt=Object.
var Mt=Object.
var St=Object.
var Pt=Object.
var Dt=Object.
var Ct=Object.
var At=Object.
var Tt=Object.
var Ot=Object.
var Rt=Object.
var Et=Object.
var Lt=Object.
var It=Object.
var zt=Object.
var Bt=Object.
var Ft=Object.
var Vt=Object.
var Wt=Object.
var Nt=Object.
var Ht=Object.
var Ut=Object.
var jt=Object.
var Kt=Object.
var Xt=Object.
var Yt=Object.
var Qt=Object.
var Zt=Object.
var Jt=Object.
var Gt=Object.
var te=Object.
var ee=Object.
var ie=Object.
var se=Object.
var ne=Object.
var oe=Object.
var ae=Object.
var re=Object.
var le=Object.
var he=Object.
var ce=Object.
var de=Object.
var ue=Object.
var pe=Object.
var fe=Object.
var ge=Object.
var me=Object.
var be=Object.
var xe=Object.
var ye=Object.
var _e=Object.
var we=Object.
var ke=Object.
var Me=Object.
var Se=Object.
var Pe=Object.
var De=Object.
var Ce=Object.
var Ae=Object.
var Te=Object.
var Oe=Object.
var Re=Object.
var Ee=Object.
var Le=Object.
var Ie=Object.
var ze=Object.
var Be=Object.
var Fe=Object.
var Ve=Object.
var We=Object.
var Ne=Object.
var He=Object.
var Ue=Object.
var je=Object.
var Ke=Object.
var Xe=Object.
var Ye=Object.
var Qe=Object.
var Ze=Object.
var Je=Object.
var Ge=Object.
var ti=Object.
var ei=Object.
var ii=Object.
var si=Object.
var ni=Object.
var oi=Object.
var ai=Object.
var ri=Object.
var li=Object.
var hi=Object.
var ci=Object.
var di=Object.
var ui=Object.
var pi=Object.
var fi=Object.
var gi=Object.
var mi=Object.
var bi=Object.
var xi=Object.
var yi=Object.
var _i=Object.
var wi=Object.
var ki=Object.
var Mi=Object.
var Si=Object.
var Pi=Object.
var Di=Object.
var Ci=Object.
var Ai=Object.
var Ti=Object.
var Oi=Object.
var Ri=Object.
var Ei=Object.
var Li=Object.
var Ii=Object.
var zi=Object.
var Bi=Object.
var Fi=Object.
var Vi=Object.
var Wi=Object.
var Ni=Object.
var Hi=Object.
var Ui=Object.
var ji=Object.
var Ki=Object.
var Xi=Object.
var Yi=Object.
var Qi=Object.
var Zi=Object.
var Ji=Object.
var Gi=Object.
var tn=Object.
var en=Object.
var in=Object.
var sn=Object.
var nn=Object.
var on=Object.
var an=Object.
var rn=Object.
var ln=Object.
var hn=Object.
var cn=Object.
var dn=Object.
var un=Object.
var pn=Object.
var fn=Object.
var gn=Object.
var mn=Object.
var bn=Object.
var xn=Object.
var yn=Object.
var _n=Object.
var wn=Object.
var kn=Object.
var Mn=Object.
var Sn=Object.
var Pn=Object.
var Dn=Object.
var Cn=Object.
var An=Object.
var Tn=Object.
var On=Object.
var Rn=Object.
var En=Object.
var Ln=Object.
var In=Object.
var zn=Object.
var Bn=Object.
var Fn=Object.
var Vn=Object.
var Wn=Object.
var Nn=Object.
var Hn=Object.
var Un=Object.
var jn=Object.
var Kn=Object.
var Xn=Object.
var Yn=Object.
var Qn=Object.
var Zn=Object.
var Jn=Object.
var Gn=Object.
var to=Object.
var eo=Object.
var io=Object.
var so=Object.
var no=Object.
var oo=Object.
var ao=Object.
var ro=Object.
var lo=Object.
var ho=Object.
var co=Object.
var do=Object.
var uo=Object.
var po=Object.
var fo=Object.
var go=Object.
var mo=Object.
var bo=Object.
var xo=Object.
var yo=Object.
var _o=Object.
var wo=Object.
var ko=Object.
var Mo=Object.
var So=Object.
var Po=Object.
var Do=Object.
var Co=Object.
var Ao=Object.
var To=Object.
var Oo=Object.
var Ro=Object.
var Eo=Object.
var Lo=Object.
var Io=Object.
var zo=Object.
var Bo=Object.
var Fo=Object.
var Vo=Object.
var Wo=Object.
var No=Object.
var Ho=Object.
var Uo=Object.
var jo=Object.
var Ko=Object.
var Xo=Object.
var Yo=Object.
var Qo=Object.
var Zo=Object.
var Jo=Object.
var Go=Object.
var tr=Object.
var er=Object.
var ir=Object.
var sr=Object.
var nr=Object.
var or=Object.
var ar=Object.
var rr=Object.
var lr=Object.
var hr=Object.
var cr=Object.
var dr=Object.
var ur=Object.
var pr=Object.
var fr=Object.
var gr=Object.
var mr=Object.
var br=Object.
var xr=Object.
var yr=Object.
var _r=Object.
var wr=Object.
var kr=Object.
var Mr=Object.
var Sr=Object.
var Pr=Object.
var Dr=Object.
var Cr=Object.
var Ar=Object.
var Tr=Object.
var Or=Object.
var Rr=Object.
var Er=Object.
var Lr=Object.
var Ir=Object.
var zr=Object.
var Br=Object.
var Fr=Object.
var Vr=Object.
var Wr=Object.
var Nr=Object.
var Hr=Object.
var Ur=Object.
var jr=Object.
var Kr=Object.
var Xr=Object.
var Yr=Object.
var Qr=Object.
var Zr=Object.
var Jr=Object.
var Gr=Object.
var tl=Object.
var el=Object.
var il=Object.
var sl=Object.
var nl=Object.
var ol=Object.
var al=Object.
var rl=Object.
var ll=Object.
var hl=Object.
var cl=Object.
var dl=Object.
var ul=Object.
var pl=Object.
var fl=Object.
var gl=Object.
var ml=Object.
var bl=Object.
var xl=Object.
var yl=Object.
var _l=Object.
var wl=Object.
var kl=Object.
var Ml=Object.
var Sl=Object.
var Pl=Object.
var Dl=Object.
var Cl=Object.
var Al=Object.
var Tl=Object.
var Ol=Object.
var Rl=Object.
var El=Object.
var Ll=Object.
var Il=Object.
var zl=Object.
var Bl=Object.
var Fl=Object.
var Vl=Object.
var Wl=Object.
var Nl=Object.
var Hl=Object.
var Ul=Object.
var jl=Object.
var Kl=Object.
var Xl=Object.
var Yl=Object.
var Ql=Object.
var Zl=Object.
var Jl=Object.
var Gl=Object.
var th=Object.
var eh=Object.
var ih=Object.
var sh=Object.
var nh=Object.
var oh=Object.
var ah=Object.
var rh=Object.
var lh=Object.
var hh=Object.
var ch=Object.
var dh=Object.
var uh=Object.
var ph=Object.
var fh=Object.
var gh=Object.
var mh=Object.
var bh=Object.
var xh=Object.
var yh=Object.
var _h=Object.
var wh=Object.
var kh=Object.
var Mh=Object.
var Sh=Object.
var Ph=Object.
var Dh=Object.
var Ch=Object.
var Ah=Object.
var Th=Object.
var Oh=Object.
var Rh=Object.
var Eh=Object.
var Lh=Object.
var Ih=Object.
var zh=Object.
var Bh=Object.
var Fh=Object.
var Vh=Object.
var Wh=Object.
var Nh=Object.
var Hh=Object.
var Uh=Object.
var jh=Object.
var Kh=Object.
var Xh=Object.
var Yh=Object.
var Qh=Object.
var Zh=Object.
var Jh=Object.
var Gh=Object.
var tc=Object.
var ec=Object.
var ic=Object.
var sc=Object.
var nc=Object.
var oc=Object.
var ac=Object.
var rc=Object.
var lc=Object.
var hc=Object.
var cc=Object.
var dc=Object.
var uc=Object.
var pc=Object.
var fc=Object.
var gc=Object.
var mc=Object.
var bc=Object.
var xc=Object.
var yc=Object.
var _c=Object.
var wc=Object.
var kc=Object.
var Mc=Object.
var Sc=Object.
var Pc=Object.
var Dc=Object.
var Cc=Object.
var Ac=Object.
var Tc=Object.
var Oc=Object.
var Rc=Object.
var Ec=Object.
var Lc=Object.
var Ic=Object.
var zc=Object.
var Bc=Object.
var Fc=Object.
var Vc=Object.
var Wc=Object.
var Nc=Object.
var Hc=Object.
var Uc=Object.
var jc=Object.
var Kc=Object.
var Xc=Object.
var Yc=Object.
var Qc=Object.
var Zc=Object.
var Jc=Object.
var Gc=Object.
var td=Object.
var ed=Object.
var id=Object.
var sd=Object.
var nd=Object.
var od=Object.
var ad=Object.
var rd=Object.
var ld=Object.
var hd=Object.
var cd=Object.
var dd=Object.
var ud=Object.
var pd=Object.
var fd=Object.
var gd=Object.
var md=Object.
var bd=Object.
var xd=Object.
var yd=Object.
var _d=Object.
var wd=Object.
var kd=Object.
var Md=Object.
var Sd=Object.
var Pd=Object.
var Dd=Object.
var Cd=Object.
var Ad=Object.
var Td=Object.
var Od=Object.
var Rd=Object.
var Ed=Object.
var Ld=Object.
var Id=Object.
var zd=Object.
var Bd=Object.
var Fd=Object.
var Vd=Object.
var Wd=Object.
var Nd=Object.
var Hd=Object.
var Ud=Object.
var jd=Object.
var Kd=Object.
var Xd=Object.
var Yd=Object.
var Qd=Object.
var Zd=Object.
var Jd=Object.
var Gd=Object.
var tu=Object.
var eu=Object.
var iu=Object.
var su=Object.
var nu=Object.
var ou=Object.
var au=Object.
var ru=Object.
var lu=Object.
var hu=Object.
var cu=Object.
var du=Object.
var uu=Object.
var pu=Object.
var fu=Object.
var gu=Object.
var mu=Object.
var bu=Object.
var xu=Object.
var yu=Object.
var _u=Object.
var wu=Object.
var ku=Object.
var Mu=Object.
var Su=Object.
var Pu=Object.
var Du=Object.
var Cu=Object.
var Au=Object.
var Tu=Object.
var Ou=Object.
var Ru=Object.
var Eu=Object.
var Lu=Object.
var Iu=Object.
var zu=Object.
var Bu=Object.
var Fu=Object.
var Vu=Object.
var Wu=Object.
var Nu=Object.
var Hu=Object.
var Uu=Object.
var ju=Object.
var Ku=Object.
var Xu=Object.
var Yu=Object.
var Qu=Object.
var Zu=Object.
var Ju=Object.
var Gu=Object.
var tp=Object.
var ep=Object.
var ip=Object.
var sp=Object.
var np=Object.
var op=Object.
var ap=Object.
var rp=Object.
var lp=Object.
var hp=Object.
var cp=Object.
var dp=Object.
var up=Object.
var pp=Object.
var fp=Object.
var gp=Object.
var mp=Object.
var bp=Object.
var xp=Object.
var yp=Object.
var _p=Object.
var wp=Object.
var kp=Object.
var Mp=Object.
var Sp=Object.
var Pp=Object.
var Dp=Object.
var Cp=Object.
var Ap=Object.
var Tp=Object.
var Op=Object.
var Rp=Object.
var Ep=Object.
var Lp=Object.
var Ip=Object.
var zp=Object.
var Bp=Object.
var Fp=Object.
var Vp=Object.
var Wp=Object.
var Np=Object.
var Hp=Object.
var Up=Object.
var jp=Object.
var Kp=Object.
var Xp=Object.
var Yp=Object.
var Qp=Object.
var Zp=Object.
var Jp=Object.
var Gp=Object.
var tm=Object.
var em=Object.
var im=Object.
var sm=Object.
var nm=Object.
var om=Object.
var am=Object.
var rm=Object.
var lm=Object.
var hm=Object.
var cm=Object.
var dm=Object.
var um=Object.
var pm=Object.
var fm=Object.
var gm=Object.
var mm=Object.
var bm=Object.
var xm=Object.
var ym=Object.
var _m=Object.
var wm=Object.
var km=Object.
var Mm=Object.
var Sm=Object.
var Pm=Object.
var Dm=Object.
var Cm=Object.
var Am=Object.
var Tm=Object.
var Om=Object.
var Rm=Object.
var Em=Object.
var Lm=Object.
var Im=Object.
var zm=Object.
var Bm=Object.
var Fm=Object.
var Vm=Object.
var Wm=Object.
var Nm=Object.
var Hm=Object.
var Um=Object.
var jm=Object.
var Km=Object.
var Xm=Object.
var Ym=Object.
var Qm=Object.
var Zm=Object.
var Jm=Object.
var Gm=Object.
var tb=Object.
var eb=Object.
var ib=Object.
var sb=Object.
var nb=Object.
var ob=Object.
var ab=Object.
var rb=Object.
var lb=Object.
var hb=Object.
var cb=Object.
var db=Object.
var ub=Object.
var pb=Object.
var fb=Object.
var gb=Object.
var mb=Object.
var bb=Object.
var xb=Object.
var yb=Object.
var _b=Object.
var wb=Object.
var kb=Object.
var Mb=Object.
var Sb=Object.
var Pb=Object.
var Db=Object.
var Cb=Object.
var Ab=Object.
var Tb=Object.
var Ob=Object.
var Rb=Object.
var Eb=Object.
var Lb=Object.
var Ib=Object.
var zb=Object.
var Bb=Object.
var Fb=Object.
var Vb=Object.
var Wb=Object.
var Nb=Object.
var Hb=Object.
var Ub=Object.
var jb=Object.
var Kb=Object.
var Xb=Object.
var Yb=Object.
var Qb=Object.
var Zb=Object.
var Jb=Object.
var Gb=Object.
var tx=Object.
var ex=Object.
var ix=Object.
var sx=Object.
var nx=Object.
var ox=Object.
var ax=Object.
var rx=Object.
var lx=Object.
var hx=Object.
var cx=Object.
var dx=Object.
var ux=Object.
var px=Object.
var fx=Object.
var gx=Object.
var mx=Object.
var bx=Object.
var xx=Object.
var yx=Object.
var _x=Object.
var wx=Object.
var kx=Object.
var Mx=Object.
var Sx=Object.
var Px=Object.
var Dx=Object.
var Cx=Object.
var Ax=Object.
var Tx=Object.
var Ox=Object.
var Rx=Object.
var Ex=Object.
var Lx=Object.
var Ix=Object.
var zx=Object.
var Bx=Object.
var Fx=Object.
var Vx=Object.
var Wx=Object.
var Nx=Object.
var Hx=Object.
var Ux=Object.
var jx=Object.
var Kx=Object.
var Xx=Object.
var Yx=Object.
var Qx=Object.
var Zx=Object.
var Jx=Object.
var Gx=Object.
var tk=Object.
var ek=Object.
var ik=Object.
var sk=Object.
var nk=Object.
var ok=Object.
var ak=Object.
var rk=Object.
var lk=Object.
var hk=Object.
var ck=Object.
var dk=Object.
var uk=Object.
var pk=Object.
var fk=Object.
var gk=Object.
var mk=Object.
var bk=Object.
var xk=Object.
var yk=Object.
var _k=Object.
var wk=Object.
var kk=Object.
var Mk=Object.
var Sk=Object.
var Pk=Object.
var Dk=Object.
var Ck=Object.
var Ak=Object.
var Tk=Object.
var Ok=Object.
var Rk=Object.
var Ek=Object.
var Lk=Object.
var Ik=Object.
var zk=Object.
var Bk=Object.
var Fk=Object.
var Vk=Object.
var Wk=Object.
var Nk=Object.
var Hk=Object.
var Uk=Object.
var jk=Object.
var Kk=Object.
var Xk=Object.
var Yk=Object.
var Qk=Object.
var Zk=Object.
var Jk=Object.
var Gk=Object.
var tl=Object.
var el=Object.
var il=Object.
var sl=Object.
var nl=Object.
var ol=Object.
var al=Object.
var rl=Object.
var ll=Object.
var hl=Object.
var cl=Object.
var dl=Object.
var ul=Object.
var pl=Object.
var fl=Object.
var gl=Object.
var ml=Object.
var bl=Object.
var xl=Object.
var yl=Object.
var _l=Object.
var wl=Object.
var kl=Object.
var Ml=Object.
var Sl=Object.
var Pl=Object.
var Dl=Object.
var Cl=Object.
var Al=Object.
var Tl=Object.
var Ol=Object.
var Rl=Object.
var El=Object.
var Ll=Object.
var Il=Object.
var zl=Object.
var Bl=Object.
var Fl=Object.
var Vl=Object.
var Wl=Object.
var Nl=Object.
var Hl=Object.
var Ul=Object.
var jl=Object.
var Kl=Object.
var Xl=Object.
var Yl=Object.
var Ql=Object.
var Zl=Object.
var Jl=Object.
var Gl
```

```
            <br style="text-family:sans-fonts;display: inline-var; inline-end: margin-bottom:0px;">Image size:</b>
            <div style="background: rgba(255, 97, 0) 0.1; border: 1px solid rgba(255, 97, 0) 0.3; padding: 8px; border-radius: 10px;">
              <b>Image Quality:</b> <span id="image_quality_value">0.8</span><br>
              <input type="range" min="0.1" max="1.0" step="0.1" value="0.8" oninput="updateQualityDisplay()">
              <br>
              <b>Image Quality:</b> <span id="image_quality_value">0.8</span><br>
              <br>
              <b>Note:</b> If you tried to re-upload files that were deleted or expired<br>
              <br>
              <button onclick="deleteAllFiles()" id="delete-all-files" disabled>Delete All Uploaded Files</b>
            <br>
            <br>
          </div>
        </div>
        <br>
        <br>
        
```

```
<meta-geometrics:size_w="65.65,0,0,0
```

```
    radius: 8.0,
    id: 8.001,
    name: "Padang school place" + stateSelect.paddingRightRect.label
  }
]
;

if (stateSelect.paddingRightRect[0]) {
  const paddingRightRect = stateSelect.paddingRightRect[0];
  log.info(`Found paddingRightRect: ${paddingRightRect.id}`);
  return {
    radius: 8.0,
    id: 8.001,
    name: "Padang university place" + stateSelect.paddingRightRect.label
  }
;
}

stateSelect.globe.pointsRemaining++;
if\stateSelect.globe.enrollmentCount++;
}
,

async function loadMapBoxRect() {
  try {
    log.info("Fetching/api/v1/spa/mbox/rectangle/pos.jsonl");
    const data = await fetch(/api/v1/spa/mbox/rectangle/pos.jsonl);
    if (!data.ok) throw new Error(`HTTP ${data.status}: ${data.statusText}`);
    stateSelect.rectangle = [];\n\nstateSelect.
    stateSelect.rectangle.push(...
    rectangle.
    );
    log.info("Loaded %d rectangle", stateSelect.rectangle.length);
    return stateSelect.rectangle;
  } catch (error) {
    console.error("Fetch rectangle endpoint failed", error);
    throw error;
  }
}

try {
  const res = await fetch(/api/v1/spa/mbox/rectangle/pos.jsonl);
  const text = \n\n\n\
```

```
const dotenvLoaded = path.join(__dirname, '.envloaded');
const logpath = path.join(__dirname, 'logs');

function*.*.*.*(*,*,*,
```

```
    const filepath = path.join(uploadDir, file.filename);
    const hash = sha256Hash(filepath);

    const exists = db.dataExists('files', 'x.sha256 IN (?)', hash && [hash]);
    if (exists) {
      deleteFiles([path.relpath(filepath, dir)], db, dbpfunction, fileo.originalname);
      results.push({name:file.originalname, error:'Already exists'});
      continue;
    }

    const newmeta = {
      sha256: hash,
      name: file.originalname,
      caption: (caption.trim() ?caption:null),
      mime: file.mimetype,
      file_path: 'uploads/'+file.filename,
      thumb_path: (thumbpath ?'thumbs/'+path.basename(thumbpath):null),
      file_type: getFileTypeFromMime(file.mimetype),
      size: file.size,
      readby: null,
      uploaded: parseInt(nowUnix ? : 0),
      deleted: null,
      hash: null,
      hashed_at: null,
      hashfail: null,
      pongdata: null,
      pongdata_at: null,
      checked_at: null,
      checkfail_at: null,
    }

    db.dataInsert('uploadmeta',newmeta);
    results.push(newmeta.id);
    deleteFiles([path.relpath(filepath, dir)], db, dbpfunction, fileo.originalname);
  }

  setcookie(req);

  if (results.length !== 1 && Array.isArray(req.body.caption.length ? 0:-1,
    return res.status(400).json({
      success: false,
      error: 'Expecting upload metadata (one file) and file(s) mismatch',
      results:results
    });
  }

  console.log(`Saw upload [${results.length}] fields: file=${req.files?req.files.length:0} [${req.files?req.files.map(f=>f.fieldname).join(','):''}]`);
  broadcast('uploadfinish',{ count: results.length, ids: results.slice(0,50) });

  res.json({
    success: true,
    message: `${results.length} uploaded${results.length>1?'s':''}`,
    count: results.length,
    ids: results.slice(0,50),
    errors: errors,
    dbpfunction:
    dir:
    uploadpath:
    uploadpathrel:
    ,error:'Upload error: '+error
  });
} catch (e) {
  res.status(500).json({ success: false, error: error.message });
}
});

// Get Uploaded Metadata
app.get('/api/uploadmeta', (req, res) => {
  try {
    const db = dataDB();

    const page = Math.max(1, parseInt(req.query.page || '1', 10));
    const limit = Math.min(50, Math.max(1, parseInt(req.query.limit || '10', 10)));
    const offset = (page-1)*limit;
    const search = (req.query.search || req.query.q||req.query.rating||'').trim();

    let dbpfunction = db.dbpfunction;
    if (!dbpfunction || !dbpfunction.exists || !db.dbtableExists db.to_pongdata,
    ,

    if (typeof dbpfunction !== 'function') dbpfunction = null;

    if (!dbpfunction) {
      dbpfunction = dbpfunction => db.dataExists('uploadmeta', dbpfunction);
    }

    const hasFunc = dbpfunction('uploadmeta');

    const where = [];
    const params = [];
    if (!hasFunc,
      where.push('(1=0 OR (x.sha256 IS NULL OR x.sha256 IN (SELECT sha256 FROM uploadmeta WHERE deleted IS NULL))');
    } else {
      where.push('(x.sha256 IS NULL OR x.sha256 IN (SELECT sha256 FROM uploadmeta WHERE deleted IS NULL))');
    }

    if (search) {
      const s = '%'+search+'%';
      where.push('(x.name LIKE ? OR x.caption LIKE ? OR x.mime LIKE ? OR x.file_path LIKE ? OR x.sha256 LIKE ?)');
      params.push(s,s,s,s,s);
    }

    const whereClause = where.length ? 'WHERE'+where.join(' AND ') :'';

    res.json({
      success: true,
      page: page,
      limit: limit,
      offset: offset,
      search: search,
      total: total,
      pages: Math.ceil(total/limit),
      items: items,
      hasMore: offset + items.length < total
    });
  } catch (error) {
    console.error('GET /api/uploadmeta error:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

// Log Memory
app.post('/api/log/memory', (req, res) => {
  try {
    const d = parseInt(req.params.id);
    if (!d||d<1) return;

    const memory = db.memoryGet('files', d);
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' });

    memory.views = (memory.views || 0) + 1;
    memory.lastview = nowUnix();
    setCookie(d);

    const m = newMemoryClass();
    m.initFromObject(memory);
    setMemoryFields();

    broadcast('memory:update', { memobjid: m.id });

    res.json({ success: true, id: memory.id, views: getNumericFieldFromMemory(m) });
  } catch (error) {
    console.error('Log memory error:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

// Reaction
app.post('/api/reaction/:memobjid', (req, res) => {
  try {
    const memobjid = parseInt(req.params.memobjid, 10);
    const { type } = req.body || {};
    const allowed = ['like','love','laugh','wow','sad','angry'];
    if (!allowed.includes(type)) return res.status(400).json({ success: false, error: 'Invalid reaction type' });

    const d = cookieGet();
    const memory = db.memoryGet('files', memobjid);
    if (!memory || memobjid !== memobjid || memobjid && memobjid !== memobjid && memobjid && memobjid !== memobjid,
      return res.status(404).json({ success: false, error: 'Memory not found' });

    const m = newMemoryClass();
    m.initFromObject(memory);
    setMemoryFields();

    broadcast('memory:update', { memobjid: m.id });

    res.json({ success: true, like: memory.like, dislike: memory.dislike, reactions: getReactionsFromMemory(m) });
  } catch (error) {
    console.error('Reaction error:', error);
    res.status(500).json({ success: false, error: e.message });
  }
});

// Comments: List
app.get('/api/comments/:memobjid', (req, res) => {
  try {
    const memobjid = parseInt(req.params.memobjid);
    if (!isFinite(memobjid,memobjid<1) {
      return res.status(400).json({ success: false, comments: [], disabled: true });
    }

    const memobjid = parseInt(req.params.memobjid, 10);
    const d = cookieGet();
    const memory = db.memoryGet('files', d && d !== memobjid && d.approved !== 1 && to.deleteAt && to.pongdata,
    if (!memory || memobjid !== memobjid || memobjid && memobjid !== memobjid && memobjid && memobjid !== memobjid,
      return res.status(404).json({ success: false, error: 'Memory not found' });

    res.json({ success: true, comments: getCommentTreeFromMemory(m), disabled: false });
  } catch (error) {
    console.error('Comments error:', error);
    res.status(500).json({ success: false, error: e.message });
  }
});

// Comments: Create
app.post('/api/comments/:memobjid', (req, res) => {
  try {
    const pongdata = getPongDataFromBody();
    if (!pongdata.enabled,
      return res.status(403).json({ success: false, error: 'Comments are disabled' });
    }

    const memobjid = parseInt(req.params.memobjid, 10);
    const d = cookieGet();
    if (!d||d<1||memobjid<1||memobjid<1) return res.status(400).json({ success: false, error: 'Invalid id' });

    if (!(name || !trimmed(name).trim() return res.status(400).json({ success: false, error: 'Name is required' });
    if (!(text || !trimmed(text).trim() return res.status(400).json({ success: false, error: 'Comment text is required' });

    if (pongdata.pongfunctionenabled &&
      !pongdata.pongfunctionenabled(req, d, pongdata,text,name,parent,null) {
      return res.status(403).json({ success: false, error: 'Comment rejected by pongfunc filter' });
    }

    const d = cookieGet();
    const memory = db.memoryGet('files', d && d !== memobjid && d.approved !== 1 && to.deleteAt && to.pongdata,
    if (!memory || memobjid !== memobjid || memobjid && memobjid !== memobjid && memobjid && memobjid !== memobjid,
      return res.status(404).json({ success: false, error: 'Memory not found' });

    const cmt = newCommentObj();

    if (parentid) {
      const parent = db.commentGet('comments', d && d !== memobjid && d.approved !== 1 && to.deleteAt && to.deleteAt,
      if (!p) return res.status(400).json({ success: false, error: 'Invalid parentid' });
    }

    const comment = {
      id: cmt.id,
      memobjid: memobjid,
      parentid: parentid ?:parent.parentid : null,
      name: String(name || 'Anon').substring(0, 50),
      text: String(text || '').substring(0, 500),
      ip: req.ip || req.connection.remoteAddress,
      ts: nowUnix(),
      deleted: false,
      hash: null,
    }

    db.commentsPutInMemory(d);
    setCookie(memory.id);

    broadcast('comment:new', { memobjid:d });

    res.json({ success: true, comment: c });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Stats
app.get('/api/stats', (req, res) => {
  try {
    const d = dataDB();
    const memory = db.memoryGet('files',d && d.approved !== 1 && to.deleteAt && to.pongdata,
    const totalsize = memory.reduce((s, m) => sum + (m.size || 0), 0);

    res.json({
      success: true,
      total: memory.length,
      totalsize: totalsize, approx length, totalsize
    });
  } catch (error) {
    console.error('Stats error:', error);
    res.status(500).json({ success: false, error: error.message });
  }
});

// ============================================================================
// API ENDPOINTS - ADMIN
// ============================================================================

// Admin Login
app.post('/api/admin/login', (req, res) => {
  try {
    const { password, passwordid } = req.body || {};
    const ip = req.ip || req.connection.remoteAddress || 'unknown';
    const pwd = String(password || '').trim();

    const admins = readAdminDB();
    const adminlist = Object.keys(admins || {});

    if (!pwd || adminlist.length === 0,
      return res.status(400).json({ success: false, error: 'Invalid credentials' });

    if (passwordid && !adminlist.includes(passwordid),
      console.log(`[ADMIN] Admin login for user:${passwordid}`);
      return res.status(400).json({ success: false, error: 'Invalid credentials' });

    if (pwd !== admin.password,
      console.log(`[ADMIN] Failed admin login for${passwordid?`:${passwordid}`:''}`);
      return res.status(401).json({ success: false, error: 'Invalid credentials' });

    const token = generateSessionToken();
    const sessionid = `${Date.now()}-${crypto.randomBytes(4).toString('hex')}`;
    const sessions = loadSessions();
    sessions.sessions = sessions.sessions ?:{ ,[token]:sessionid, ip: req.ip,
      created: nowUnix(), lastseen: nowUnix(), passwordid: passwordid||null, user: admin.id
    };
    saveSessions(sessions);

    addClient(sid, 'login', {});

    res.json({
      success: true,
      token: token,
      sessionid: sessionid,
      passwordid: passwordid||null,
      ,user: { id: admin.id, name: admin.name, mrkv: admin.mrkv, permissions: admin.permissions }
    });
  } catch (e) {
    res.status(500).json({ success: false, error: e.message });
  }
});

// Verify Admin
app.get('/api/admin/verify', (req, res) => {
  try {
    const token = req.headers['x-admin-token'] || req.query.token || req.cookies?.admin_token,
    const session = getSessionFromToken(token);
    if (!session || !isSessionValid(session,token,req.ip,req.connection?.remoteAddress,req.header('User-Agent'),req.header('Accept-Language'),req.header('Accept-Encoding'),req.header('Sec-Ch-Ua'),req.header('Sec-Ch-Ua-Platform'),req.header('Sec-Ch-Ua-Mobile'),req.header('Sec-Fetch-Site'),req.header('Sec-Fetch-Mode'),req.header('Sec-Fetch-Dest'),req.header('Sec-Fetch-User'),req.header('Upgrade-Insecure-Requests'),req.header('Cache-Control'),req.header('Pragma'),req.header('Connection'),req.header('Accept'),req.header('Accept-Charset'),req.header('Accept-Datetime'),req.header('Accept-Encoding'),req.header('Accept-Language'),req.header('Accept-Ranges'),req.header('Accept-Range'),req.header('Accept-Range-Unit'),req.header('Accept-Range-Value'),req.header('Accept-Range-Value-Unit'),req.header('Accept-Range-Value-Value'),req.header('Accept-Range-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value'),req.header('Accept-Range-Value-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value-Value'),req.header('Accept-Range-Value-Value-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value-Value-Value'),req.header('Accept-Range-Value-Value-Value-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Unit'),req.header('Accept-Range-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value-Value'),req.header('Accept-Range-Value
```

```
    if (!memory) return res.status(404).json({ success: false, error: 'Memory not found' });

    const fp = path.join(uploadsDir, memory.file_path);
    if (fs.existsSync(fp) && fs.unlinkSync(fp);

    db.run('DELETE FROM memories WHERE id = ?', [req.params.id]);
    db.run('DELETE FROM tags WHERE memory_id = ?', [req.params.id]);
    saveDB();

    auditLog('admin', 'DELETE', 'memories', req.params.id);
    broadcast('memory:deleted', { id: req.params.id });

    res.json({ success: true, message: 'Forgotten permanently' });
  } catch (err) { res.status(500).json({ success: false, error: err.message }); }
});

// Bulk actions
app.post('/api/admin/bulk', (req, res) => {
  try {
    if (!isAdmin(req, res)) return;
    const { action, ids } = req.body;
    if (!Array.isArray(ids || [])) return res.status(400).json({ success: false, error: 'Invalid ids' });

    const placeholders = ids.map(() => '?').join(', ');
    if (action === 'delete') {
      const files = db.prepare('SELECT file_path FROM memories WHERE id IN (' + placeholders + ')').all(...ids);
      for (const f of files) {
        const fp = path.join(uploadsDir, f.file_path);
        if (fs.existsSync(fp) && fs.unlinkSync(fp);
      }
      db.run('DELETE FROM memories WHERE id IN (' + placeholders + ')', ...ids);
      db.run('DELETE FROM tags WHERE memory_id IN (' + placeholders + ')', ...ids);
      saveDB();
      auditLog('admin', 'BULK_DELETE', 'memories', null, { count: ids.length });
      broadcast('memory:bulk', { action: 'delete', ids });
    } else if (action === 'lock') {
      db.run('UPDATE memories SET locked = 1 WHERE id IN (' + placeholders + ')', ...ids);
      saveDB();
      auditLog('admin', 'BULK_LOCK', 'memories', null, { count: ids.length });
      broadcast('memory:bulk', { action: 'lock', ids });
    } else if (action === 'unlock') {
      db.run('UPDATE memories SET locked = 0 WHERE id IN (' + placeholders + ')', ...ids);
      saveDB();
      auditLog('admin', 'BULK_UNLOCK', 'memories', null, { count: ids.length });
      broadcast('memory:bulk', { action: 'unlock', ids });
    } else if (action === 'archive') {
      db.run('UPDATE memories SET archived = 1 WHERE id IN (' + placeholders + ')', ...ids);
      saveDB();
      auditLog('admin', 'BULK_ARCHIVE', 'memories', null, { count: ids.length });
      broadcast('memory:bulk', { action: 'archive', ids });
    } else if (action === 'unarchive') {
      db.run('UPDATE memories SET archived = 0 WHERE id IN (' + placeholders + ')', ...ids);
      saveDB();
      auditLog('admin', 'BULK_UNARCHIVE', 'memories', null, { count: ids.length });
      broadcast('memory:bulk', { action: 'unarchive', ids });
    } else if (action === 'tag') {
      const { tag } = req.body;
      if (!tag || typeof tag !== 'string') return res.status(400).json({ success: false, error: 'Invalid tag' });
      const now = new Date().toISOString();
      for (const id of ids) {
        db.run('INSERT OR IGNORE INTO tags (memory_id, tag, created_at) VALUES (?, ?, ?)', [id, tag, now]);
      }
      saveDB();
      auditLog('admin', 'BULK_TAG', 'memories', null, { count: ids.length, tag });
      broadcast('memory:bulk', { action: 'tag', ids, tag });
    } else if (action === 'untag') {
      const { tag } = req.body;
      if (!tag || typeof tag !== 'string') return res.status(400).json({ success: false, error: 'Invalid tag' });
      db.run('DELETE FROM tags WHERE memory_id IN (' + placeholders + ') AND tag = ?', [...ids, tag]);
      saveDB();
      auditLog('admin', 'BULK_UNTAG', 'memories', null, { count: ids.length, tag });
      broadcast('memory:bulk', { action: 'untag', ids, tag });
    } else if (action === 'change_type') {
      const { memory_type } = req.body;
      if (!memory_type || !['text', 'image', 'audio', 'video', 'file', 'note'].includes(memory_type)) {
        return res.status(400).json({ success: false, error: 'Invalid memory_type' });
      }
      db.run('UPDATE memories SET memory_type = ? WHERE id IN (' + placeholders + ')', [memory_type, ...ids]);
      saveDB();
      auditLog('admin', 'BULK_CHANGE_TYPE', 'memories', null, { count: ids.length, memory_type });
      broadcast('memory:bulk', { action: 'change_type', ids, memory_type });
    } else {
      return res.status(400).json({ success: false, error: 'Unknown action' });
    }
    res.json({ success: true, count: ids.length });
  } catch (err) { res.status(500).json({ success: false, error: err.message }); }
});

// Admin Batch Upload (Image Backfill, Video Ingest, Profile Photos, and Item-scoped)
// Accepts multiple files, optional JSON metadata, optional tags, optional item_id.
app.post('/api/admin/upload', (req, res) => {
  try {
    if (!isAdmin(req, res)) return;
    if (!uploadsDir) {
      return res.status(500).json({ success: false, error: 'Uploads directory not configured' });
    }

    const { files, captions, tags, metadatas, item_id } = req.body;
    const fileArr = Array.isArray(files) ? files : [];

    if (!fileArr || fileArr.length === 0) {
      return res.status(400).json({ success: false, error: 'No files uploaded' });
    }
    if (fileArr.length > 50) {
      return res.status(400).json({ success: false, error: 'Too many files (max 50)' });
    }
    if (fileArr.[0].base64.length > 15 * 1024 * 1024) {
      return res.status(400).json({ success: false, error: 'Files too large' });
    }

    const db = getDB();
    const now = new Date();
    const uploadedIds = metadatas && metadatas.length > 0 ? [] : [];

    fileArr.forEach((f, i) => {
      const ext = (f.name || '').split('.').pop() || 'bin';
      const safeExt = ext.replace(/[^a-z0-9]/gi, '').slice(0, 5);
      const randomPart = Math.random.toFixed(4).slice(2, 6);
      const fileName = now.getTime() + '-' + randomPart + '.' + safeExt;
      const filePath = path.join(uploadsDir, fileName);

      // Decode base64 and write to disk
      const buffer = Buffer.from(f.base64, 'base64');
      fs.writeFileSync(filePath, buffer);

      const memory = {
        id: uuid(),
        text_content: null,
        file_name: f.name,
        caption: captions && captions[i] ? captions[i].slice(0, 500) :
        file_path: fileName,
        file_size: buffer.length,
        mime_type: f.mime || 'application/octet-stream',
        memory_type: getMemoryType(f.mime||ext),
        tags: '[]',
        pinned: false,
        locked: false,
        favorite: false,
        archived: false,
        deleted: false,
        deleted_at: null,
        created_at: now,
        updated_at: now,
        expires_at: null,
        item_id: item_id || null,
      };

      db.memories.push(memory);
      db.insertRowIntoIndex(db.memories, memory);

      uploadedIds.push(memory.id);

      if (metadatas && metadatas[i] && metadatas[i].tags && Array.isArray(metadatas[i].tags)) {
        const tagsArr = metadatas[i].tags.slice(0, 20);
        const insertTag = db.prepare('INSERT OR IGNORE INTO tags (memory_id, tag, created_at) VALUES (?, ?, ?)');
        tagsArr.forEach(t => {
          if (typeof t === 'string' && t.length > 0 && t.length < 50) {
            insertTag.run(memory.id, t.trim(), now);
          }
        });
        insertTag.free();
      }
    });

    saveDB();
    auditLog('admin', 'UPLOAD', 'memories', null, { count: uploadedIds.length, item_id: item_id || null });
    broadcast('memory:bulk', { action: 'upload', ids: uploadedIds });

    console.log('[Admin Upload] ' + uploadedIds.length + ' file(s) ingested' + (item_id ? ' for item ' + item_id : ''));

    res.json({ success: true, count: uploadedIds.length, ids: uploadedIds });
  } catch (err) {
    console.error('[Admin Upload] Error:', err.stack);
    res.status(500).json({ success: false, error: err.message });
  }
});

// DELETE memory variations
app.delete('/api/admin/memories/:id', (req, res) => {
  try {
    if (!isAdmin(req, res)) return;
    if (!uploadsDir) {
      return res.status(500).json({ success: false, error: 'Uploads directory not configured' });
    }

    const id = req.params.id;
    const mem = db.prepare('SELECT file_path FROM memories WHERE id = ?').get(id);
    if (!mem) return res.status(404).json({ success: false, error: 'Memory not found' });

    const fp = path.join(uploadsDir, mem.file_path);
    if (fs.existsSync(fp) && fs.unlinkSync(fp);

    db.run('DELETE FROM memories WHERE id = ?', [id]);
    db.run('DELETE FROM tags WHERE memory_id = ?', [id]);
    saveDB();

    auditLog('admin', 'DELETE', 'memories', id);
    broadcast('memory:deleted', { id });

    res.json({ success: true, message: 'Memory permanently deleted' });
  } catch (err) { res.status(500).json({ success: false, error: err.message }); }
});

// Replace file
app.post('/api/admin/memories/:id/replace-file', upload.single('file'), (req, res) => {
  try {
    if (!isAdmin(req, res)) return;
    if (!uploadsDir) {
      return res.status(500).json({ success: false, error: 'Uploads directory not configured' });
    }

    const id = req.params.id;
    const mem = db.prepare('SELECT * FROM memories WHERE id = ?').get(id);
    if (!mem) return res.status(404).json({ success: false, error: 'Item not found' });

    const f = req.file;
    if (!f) {
      return res.status(400).json({ success: false, error: 'No file uploaded' });
    }

    const buffer = f.buffer;
    if (buffer.length > 15 * 1024 * 1024) {
      return res.status(400).json({ success: false, error: 'File too large (max 15MB)' });
    }

    const ext = (f.name || '').split('.').pop() || 'bin';
    const safeExt = ext.replace(/[^a-z0-9]/gi, '').slice(0, 5);
    const randomPart = Math.random.toFixed(4).slice(2, 6);
    const fileName = now.getTime() + '-' + randomPart + '.' + safeExt;
    const filePath = path.join(uploadsDir, fileName);

    fs.writeFileSync(filePath, buffer);

    const oldPath = mem.file_path ? path.join(uploadsDir, mem.file_path);
    if (oldPath && oldPath !== filePath && fs.existsSync(oldPath)) {
      try { fs.unlinkSync(oldPath); }
      catch (e) { console.warn('[Admin] Failed to delete old file:', e.message); }
    }

    const newMime = f.mime || 'application/octet-stream';
    const newType = getMemoryType(newMime);

    db.run('UPDATE memories SET file_name = ?, file_path = ?, file_size = ?, mime_type = ?, memory_type = ?, updated_at = ? WHERE id = ?',
           [f.name, fileName, buffer.length, newMime, newType, now, id]);
    saveDB();

    auditLog('admin', 'REPLACE_FILE', 'memories', id, { old_file: mem.file_name, new_file: f.name, new_size: buffer.length });
    broadcast('memory:updated', { id, memory: db.memoryRowToObject(db.memories, id) });

    console.log('[Admin] Replaced file for ' + id + ': ' + (mem.file_name || '(none)') + ' → ' + f.name + ' (' + buffer.length + ' bytes)');

    res.json({ success: true, file_name: f.name, file_size: buffer.length, mime_type: newMime, memory_type: newType });
  } catch (err) { res.status(500).json({ success: false, error: err.message }); }
});

// Download all approved as ZIP
app.get('/api/admin/download.zip', (req, res) => {
  try {
    if (!isAdmin(req, res)) return;
    const { tags = '', exp_days = '', exp_hours = '', exp_minutes = '', password = '', max_downloads = '', ttl = '' } = req.query;
    if (!archiverModule||!fs||!uploadsDir) return res.status(500).json({ success: false, error: 'Unavailable' });

    const db = getDB();
    const approvedCount = db.prepare('SELECT COUNT(*) FROM memories WHERE pinned = 1 AND deleted = 0').get();
    if (!approvedCount || approvedCount.count === 0) return res.status(404).json({ success: false, error: 'No approved memories to download' });

    const MAX = 500 * 1024 * 1024;
    const totalSize = db.prepare('SELECT COALESCE(SUM(file_size), 0) FROM memories WHERE pinned = 1 AND deleted = 0').get();
    if (totalSize && totalSize.sum > MAX) return res.status(413).json({ success: false, error: 'Too large (max 500MB)' });

    const tagFilter = tags ? tags.split(',').map(t => t.trim()).filter(Boolean) : [];
    const expDays = parseFloat(exp_days) || 0, expHours = parseFloat(exp_hours) || 0, expMinutes = parseFloat(exp_minutes) || 0;
    const hasExpiry = expDays > 0 || expHours > 0 || expMinutes > 0;
    const expiresAt = hasExpiry ? new Date(Date.now() + expDays*86400000 + expHours*3600000 + expMinutes*60000).toISOString() : null;
    const maxDl = parseInt(max_downloads) > 0 ? parseInt(max_downloads) : null;
    const ttlHours = parseFloat(ttl) > 0 ? parseFloat(ttl) : null;

    res.setHeader('Content-Type', 'application/zip');
    res.setHeader('Content-Disposition', 'attachment; filename="memory-bank-' + new Date().toISOString().slice(0,10).replace(/-/g,'') + '.zip"');

    const archive = archiverModule('zip', { zlib: { level: 6 } });
    archive.on('error', (err) => { console.error('[Admin] Download error:', err.message); if (!res.headersSent) res.status(500).end(); });
    archive.pipe(res);

    const rows = db.prepare('SELECT * FROM memories WHERE pinned = 1 AND deleted = 0 ORDER BY created_at DESC').all();
    const usedNames = {};
    for (const m of rows) {
      if (tagFilter.length > 0 && !tagFilter.some(t => (m.tags||'').toLowerCase().includes(t.toLowerCase()))) continue;
      if (m.file_path && m.file_name) {
        const fp = path.join(uploadsDir, m.file_path);
        if (fs.existsSync(fp) && fs.statSync(fp).size > 0) {
          let name = m.file_name;
          if (usedNames[name]) { const ext = name.lastIndexOf('.'); name = (ext > 0 ? name.slice(0,ext) : name) + '-' + m.id.slice(0,8) + (ext > 0 ? name.slice(ext) : ''); }
          usedNames[m.file_name] = 1;
          archive.file(fp, { name: 'media/' + name });
        }
      }
      if (m.text_content && m.text_content.trim.length > 0) {
        const name = 'text/' + (m.id||'').slice(0,8) + '.txt';
        archive.append(m.text_content, { name: name });
      }
    }

    const manifest = JSON.stringify({ exported_at: new Date().toISOString(), count: rows.length, filters: { tags: tagFilter, exp_days: expDays, exp_hours: expHours, exp_minutes: expMinutes, password: password ? '(set)' : null, max_downloads: maxDl, ttl: ttlHours } }, null, 2);
    archive.append(manifest, { name: 'manifest.json' });

    archive.finalize();
    auditLog('admin', 'DOWNLOAD', 'memories', null, { count: rows.length, tags: tagFilter, exp_days: expDays, exp_hours: expHours, exp_minutes: expMinutes, password: password ? '(set)' : null, max_downloads: maxDl, ttl: ttlHours });
    broadcast('memory:downloaded', { count: rows.length, tags: tagFilter });
  } catch (err) { console.error('[Admin] Download error:', err.stack); if (!res.headersSent) res.status(500).end(); }
});

// Change max password
app.post('/api/admin/change-password', (req, res) => {
  try {
    if (!isAdmin(req, res)) return;
    const db = getDB();
    const { current, new: newPass } = req.body || {};
    if (!current||typeof current !== 'string') return res.status(400).json({ success: false, error: 'Missing current' });
    if (!newPass||typeof newPass !== 'string') return res.status(400).json({ success: false, error: 'Missing new password' });
    if (newPass.length < 8) return res.status(400).json({ success: false, error: 'Minimum 8 characters' });

    const admin = db.prepare('SELECT * FROM users WHERE role = ? LIMIT 1').get('admin');
    if (!admin||!admin.password_hash) return res.status(400).json({ success: false, error: 'No admin password set' });

    if (!verifyPassword(current, admin.password_hash)) return res.status(401).json({ success: false, error: 'Wrong password' });

    db.run('UPDATE users SET password_hash = ? WHERE id = ?', [hashPassword(newPass), admin.id]);
    saveDB();
    auditLog('admin', 'PASSWORD_CHANGE', 'users', admin.id);
    broadcast('admin:password_changed', { at: new Date().toISOString() });

    console.log('[Admin] Admin password changed');

    res.json({ success: true, message: 'Updated' });
  } catch (err) { res.status(500).json({ success: false, error: err.message }); }
});

// Stats management - GET
app.get('/api/admin/stats/settings', (req, res) => {
  try {
    if (!isAdmin(req, res)) return;
    const db = getDB();
    if (!db||!db.settings) return res.status(500).json({ success: false, error: 'Unavailable' });

    const s = db.settings;
    res.json({
      success: true,
      settings: {
        enabled: !!(s.stats_enabled ||
        date: s.date,
        time: s.time,
        tz: s.tz,
        dayOfWeek: s.dayOfWeek,
        dayOfMonth: s.dayOfMonth,
        intervalHours: s.intervalHours,
        intervalMinutes: s.intervalMinutes,
        lastRunAt: s.lastRunAt,
      }
    });
  } catch (err) { res.status(500).json({ success: false, error: err.message }); }
});

// Stats management - PUTS
app.put('/api/admin/stats/settings', (req, res) => {
  try {
    if (!isAdmin(req, res)) return;
    const db = getDB();
    if (!db||!db.settings) return res.status(500).json({ success: false, error: 'Unavailable' });

    const { enabled, password, time, date, tz, dayOfWeek, dayOfMonth, intervalHours, intervalMinutes } = req.body || {};
    if (password||typeof password !== 'string') return res.status(400).json({ success: false, error: 'Wrong password' });
    if (!verifyPassword(password, db.settings.password_hash)) return res.status(401).json({ success: false, error: 'Wrong password' });

    const s = db.settings;
    if (typeof enabled !== 'boolean') s.stats_enabled = !!enabled;
    if (typeof time !== 'string'||/^\d{1,2}:\d{1,2}$/.test(time)) s.time = time;
    if (typeof date !== 'string'||/^\d{4}-\d{1,2}-\d{1,2}$/.test(date)) s.date = date;
    if (typeof tz !== 'string'||tz.length < 50) s.tz = tz;
    if (typeof dayOfWeek !== 'number'||(dayOfWeek >= 0 && dayOfWeek <= 6) s.dayOfWeek = dayOfWeek;
    if (typeof dayOfMonth !== 'number'||(dayOfMonth >= 1 && dayOfMonth <= 31) s.dayOfMonth = dayOfMonth;
    if (typeof intervalHours !== 'number'||intervalHours >= 0) s.intervalHours = intervalHours;
    if (typeof intervalMinutes !== 'number'||intervalMinutes >= 0) s.intervalMinutes = intervalMinutes;

    saveDB();
    auditLog('admin', 'STATS_SETTINGS', 'settings', null);
    broadcast('stats:settings', { enabled: !!s.stats_enabled });

    console.log('[Admin] Stats settings updated');

    res.json{ success: true, message: 'Updated' });
  } catch (err) { res.status(500).json{ success: false, error: err.message }); }
});

// Stats management - Run now
app.post('/api/admin/stats/run-now', (req, res) => {
  try {
    if (!isAdmin(req, res)) return;
    const db = getDB();
    if (!db||!db.settings) return res.status(500).json({ success: false, error: 'Unavailable' });

    const s = db.settings;
    if (!s.stats_enabled) return res.status(400).json{ success: false, error: 'Stats not enabled' });

    const { sendStatsNow } = require('./stats');
    if (!sendStatsNow||typeof sendStatsNow !== 'function') return res.status(500).json{ success: false, error: 'Stats module not available' });

    sendStatsNow(db, s).then(() => {
      s.lastRunAt = new Date().toISOString();
      saveDB();
      broadcast('stats:sent', { at: s.lastRunAt });
      console.log('[Admin] Stats sent');
      res.json{ success: true, message: 'Sent' });
    }).catch(err => {
      console.error('[Admin] Stats send error:', err.message);
      res.status(500).json{ success: false, error: err.message });
    });
  } catch (err) { res.status(500).json{ success: false, error: err.message }); }
});

// Export map
app.get('/api/admin/exports/:id', (req, res) => {
  try {
    if (!isAdmin(req, res)) return;
    const db = getDB();
    if (!db||!db.exports) return res.status(500).json{ success: false, error: 'Unavailable' });

    const id = req.params.id;
    const e = db.exports.find(x => x.id === id);
    if (!e) return res.status(404).json{ success: false, error: 'Not found' });

    const dir = path.join(uploadsDir, 'exports');
    const fp = path.join(dir, e.file_name);
    if (!fp.startsWith(dir + path.sep)) return res.status(400).json{ success: false, error: 'Invalid path' });

    res.setHeader('Content-Type', e.mime_type||'application/zip');
    res.setHeader('Content-Disposition', 'attachment; filename="' + (e.file_name||'export.zip').replace/[\r\n"]/g, '_') + '"');
    res.setHeader('Cache-Control', 'no-store');

    const stream = require('fs').createReadStream(fp);
    stream.on('error', (err) => {
      console.error('[Admin] Export stream error:', err.message);
      if (!res.headersSent) res.status(500).end();
      else res.destroy();
    });
    stream.pipe(res);
  } catch (err) { res.status(500).end(); }
});

// ═══════════════════════════════════════════════════════════════
// CONTRIBUTIONS API
// ═══════════════════════════════════════════════════════════════

// GET all contributions (public)
app.get('/api/contributions', (req, res) => {
  try {
    if (!isAdmin(req, res)) return;
    const db = getDB();
    res.json({ success: true, contributions: db.contributions || [] });
  } catch (err) {
    res.status(500).json{ success: false, contributions: db.contributions || [] });
  }
});

// GET single contribution (public)
app.get('/api/contributions/:id', (req, res) => {
  try {
    const id = parseInt(req.params.id, 10);
    const db = getDB();
    const c = (db.contributions || []).find(x => x.id === id);
    if (!c) return res.status(404).json{ success: false, error: 'Not found' });
    res.setHeader('Cache-Control', 'no-store');
    res.json{ success: true, contribution: c });
  } catch (err) {
    res.status(500).json{ success: false, error: err.message });
  }
});

// Create contribution (admin)
app.post('/api/admin/contributions', (req, res) => {
  try {
    if (!isAdmin(req, res)) return;
    const db = getDB();
    if (!db||!db.contributions) {
      return res.status(500).json{ success: false, error: 'Table not ready' });
    }

    const { text, image_data, image_mime, image_name, tags, pinned, expiry_hours } = req.body || {};
    const hasImg = !!(image_data && image_mime);
    if (!(text||'').trim.length && !hasImg) {
      return res.status(400).json{ success: false, error: 'Text and image required' });
    }

    const MAX = 2 * 1024*1024*1024;
    if (image_data && image_data.length > MAX) {
      return res.status(413).json{ success: false, error: 'Too large (max 2GB)' });
    }

    const id = uuid();
    const now = new Date().toISOString();
    const tagsJ = JSON.stringify(Array.isArray(tags) ? tags.slice(0, 20).map(t => String(t).slice(0, 50)) : []);
    const exp = (expiry_hours > 0 && expiry_hours <= 8760) ? new Date(Date.now() + expiry_hours*3600000).toISOString() : null;

    db.run('INSERT INTO contributions (id, text_content, image_data, image_mime, image_name, tags, pinned, expiry_hours, expiresAt, created_at, updated_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)',
           [id, (text||'').trim, image_data||null, image_mime||null, image_name||null, tagsJ, pinned ? 1 : 0, expiry_hours||null, exp, now, now]);
    saveDB();

    auditLog('admin', 'CREATE', 'contributions', id, { hasImg, tags: tagsJ, pinned: !!pinned, expiry_hours: expiry_hours||null });
    broadcast('contribution:created', { id, hasImg, tags: tagsJ, pinned: !!pinned, expiry_hours: expiry_hours||null, expiresAt: exp, created_at: now, updated_at: now });

    console.log('[Contribution] Created ' + id + (hasImg ? ' (image)' : ' (text)'));

    res.setHeader('Cache-Control', 'no-store');
    res.status(201).json{ success: true, id, created_at: now });
  } catch (err) {
    console.error('[Contribution] Create error:', err.stack);
    res.status(500).json{ success: false, error: err.message });
  }
});

// Update contribution (admin)
app.put('/api/admin/contributions/:id', (req, res) => {
  try {
    if (!isAdmin(req, res)) return;
    const db = getDB();

    const id = parseInt(req.params.id, 10);
    const mem = db.contributions||[].find(x => x.id === id);
    if (!mem) return res.status(404).json{ success: false, error: 'Not found' });

    const { text, image_data, image_mime, image_name, tags, pinned, expiry_hours, remove_image } = req.body || {};
    if (text !== undefined && (text||'').trim.length > 0) mem.text_content = (text||'').trim.slice(0, 5000);
    if (remove_image) { mem.image_data = null; mem.image_mime = null; mem.image_name = null; }
    else if (image_data && image_mime) { mem.image_data = image_data; mem.image_mime = image_mime; mem.image_name = image_name||null; }
    if (Array.isArray(tags)) mem.tags = JSON.stringify(tags.slice(0, 20).map(t => String(t).slice(0, 50)));
    if (typeof pinned === 'boolean') mem.pinned = pinned ? 1 : 0;
    if (expiry_hours !== undefined) { mem.expiry_hours = expiry_hours||null; mem.expiresAt = (expiry_hours > 0 && expiry_hours <= 8760) ? new Date(Date.now() + expiry_hours*3600000).toISOString() : null; }
    mem.updated_at = new Date().toISOString();

    saveDB();
    auditLog('admin', 'UPDATE', 'contributions', id, { remove_image: !!remove_image, pinned: !!pinned, expiry_hours: expiry_hours||null });
    broadcast('contribution:updated', { id, updated_at: mem.updated_at });

    console.log('[Contribution] Updated ' + id);

    res.setHeader('Cache-Control', 'no-store');
    res.status(200).json{ success: true, id, updated_at: mem.updated_at });
  } catch (err) {
    console.error('[Contribution] Update error:', err.stack);
    res.status(500).json{ success: false, error: err.message });
  }
});

// Delete contribution (admin)
app.delete('/api/admin/contributions/:id', (req, res) => {
  try {
    if (!isAdmin(req, res)) return;
    const db = getDB();

    const id = parseInt(req.params.id, 10);
    const mem = db.contributions||[].find(x => x.id === id);
    if (!mem) return res.status(404).json{ success: false, error: 'Not found' });

    db.contributions = db.contributions.filter(x => x.id !== id);
    saveDB();

    auditLog('admin', 'DELETE', 'contributions', id, { hasImg: !!mem.image_data, tags: mem.tags, pinned: !!mem.pinned });
    broadcast('contribution:deleted', { id, deleted_at: new Date().toISOString() });

    console.log('[Contribution] Deleted ' + id);

    res.setHeader('Cache-Control', 'no-store');
    res.status(200).json{ success: true, id, deleted_at: new Date().toISOString() });
  } catch (err) {
    res.status(500).json{ success: false, error: err.message });
  }
});

// Update contribution (admin)
app.put('/api/admin/contributions/:id', (req, res) => {
  try {
    if (!isAdmin(req, res)) return;
    const db = getDB();

    const id = parseInt(req.params.id, 10);
    const mem = db.contributions||[].find(x => x.id === id);
    if (!mem) return res.status(404).json{ success: false, error: 'Not found' });

    const { text, image_data, image_mime, image_name, tags, pinned, expiry_hours, remove_image } = req.body || {};
    if (text !== undefined && (text||'').trim.length > 0) mem.text_content = (text||'').trim.slice(0, 5000);
    if (remove_image) { mem.image_data = null; mem.image_mime = null; mem.image_name = null; }
    else if (image_data && image_mime) { mem.image_data = image_data; mem.image_mime = image_mime; mem.image_name = image_name||null; }
    if (Array.isArray(tags)) mem.tags = JSON.stringify(tags.slice(0, 20).map(t => String(t).slice(0, 50)));
    if (typeof pinned === 'boolean') mem.pinned = pinned ? 1 : 0;
    if (expiry_hours !== undefined) { mem.expiry_hours = expiry_hours||null; mem.expiresAt = (expiry_hours > 0 && expiry_hours <= 8760) ? new Date(Date.now() + expiry_hours*3600000).toISOString() : null; }
    mem.updated_at = new Date().toISOString();

    saveDB();
    auditLog('admin', 'UPDATE', 'contributions', id, { remove_image: !!remove_image, pinned: !!pinned, expiry_hours: expiry_hours||null });
    broadcast('contribution:updated', { id, updated_at: mem.updated_at });

    console.log('[Contribution] Updated ' + id);

    res.setHeader('Cache-Control', 'no-store');
    res.status(200).json{ success: true, id, updated_at: mem.updated_at });
  } catch (err) {
    res.status(500).json{ success: false, error: err.message });
  }
});
```

```
  ok: [],
  errors: [{ message: "Unauthorized" }]
}

// Delete computation (admin)
app.post("/api/admin/computations/:id", (req, res) => {
  const auth = requireAdmin(req, res);
  if (!auth) return;

  const id = parseInt(req.params.id, 10);
  const data = readDataSafe();
  const before = data.computations.length;
  data.computations = data.computations.filter(c => c.id !== id);
  if (data.computations.length === before) return res.status(404).json({ ok: false, error: "Not found" });

  data.computations.sort((a, b) =>
    (a.order ?? 0) - (b.order ?? 0));

  addAuditLog(req, auth.id, "delete_computation", { id });
  writeDataSafe(data);
  res.set("Cache-Control", "no-store");
  res.set("Pragma", "no-cache");
  res.set("Expires", "0");
  res.set("Surrogate-Control", "no-store");
  res.set("CDN-Cache-Control", "no-store");
  res.set("Vary", "Cookie, Authorization");
  res.set("Last-Modified", new Date().toUTCString());
  res.set("ETag", "W/\"" + Date.now() + "\"");
  res.set("X-Accel-Buffering", "no");
  res.set("Surrogate-Key", "computations");
  res.set("Cache-Tag", "computations");
  res.set("Cloudflare-CDN-Cache-Control", "no-store");
  res.set("Pragma-Directive", "no-cache");
  res.set("Expires-Directive", "0");
  res.set("Cache-Control-Directive", "no-store, no-cache, must-revalidate, max-age=0, s-maxage=0, private");
  res.set("Vary-Extended", "Cookie, Authorization, Accept, Accept-Encoding");
  res.set("Cache-Control-Extended", "no-store, no-cache, must-revalidate, max-age=0, s-maxage=0, private, no-transform");
  res.set("Cache-Control-Final", "no-store, no-cache, must-revalidate, max-age=0, s-maxage=0, private, no-transform, immutable");
  res.set("Cache-Control-Ultimate", "no-store, no-cache, must-revalidate, max-age=0, s-maxage=0, private, no-transform, immutable, stale-while-revalidate=0");
  res.set("Cache-Control-Supreme", "no-store, no-cache, must-revalidate, max-age=0, s-maxage=0, private, no-transform, immutable, stale-while-revalidate=0, stale-while-revalidate=0");
  res.set("Cache-Control-Absolute", "no-store, no-cache, must-revalidate, max-age=0, s-maxage=0, private, no-transform, immutable, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0");
  res.set("Cache-Control-Eternal", "no-store, no-cache, must-revalidate, max-age=0, s-maxage=0, private, no-transform, immutable, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0");
  res.set("Cache-Control-Infinite", "no-store, no-cache, must-revalidate, max-age=0, s-maxage=0, private, no-transform, immutable, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0");
  res.set("Cache-Control-Infinity", "no-store, no-cache, must-revalidate, max-age=0, s-maxage=0, private, no-transform, immutable, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0");
  res.set("Cache-Control-Infinity-Infinity", "no-store, no-cache, must-revalidate, max-age=0, s-maxage=0, private, no-transform, immutable, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0");
  res.set("Cache-Control-Infinity-Infinity-Infinity", "no-store, no-cache, must-revalidate, max-age=0, s-maxage=0, private, no-transform, immutable, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0");
  res.set("Cache-Control-Infinity-Infinity-Infinity-Infinity", "no-store, no-cache, must-revalidate, max-age=0, s-maxage=0, private, no-transform, immutable, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0");
  res.set("Cache-Control-Infinity-Infinity-Infinity-Infinity-Infinity", "no-store, no-cache, must-revalidate, max-age=0, s-maxage=0, private, no-transform, immutable, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0");
  res.set("Cache-Control-Infinity-Infinity-Infinity-Infinity-Infinity-Infinity", "no-store, no-cache, must-revalidate, max-age=0, s-maxage=0, private, no-transform, immutable, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0");
  res.set("Cache-Control-Infinity-Infinity-Infinity-Infinity-Infinity-Infinity-Infinity", "no-store, no-cache, must-revalidate, max-age=0, s-maxage=0, private, no-transform, immutable, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0, stale-while-revalidate=0
```

```
    if (typeof slug !== 'string') {
      return [{ phase: 'slug-type', ok: false, error: 'slug must be a string' }];
    }
    const checks = [];
    checks.push({ phase: 'slug-length', ok: slug.length > 0 && slug.length <= 128, detail: slug.length });
    var bad = /[^a-z0-9-]/.test(slug);
    checks.push({ phase: 'slug-charset', ok: !bad, detail: bad ? 'contains illegal characters' : null });
    var dashes = (slug.match(/-/g) || [].length);
    checks.push({ phase: 'slug-dashes', ok: dashes <= 4, detail: dashes });
    return checks;
  }

  function _mk(){ var bytes = crypto.randomBytes(8); return bytes.toString('hex'); }

  function _hmacHex(secret, payload){
    return crypto.createHmac('sha256', secret).update(payload, 'utf8').digest('hex');
  }

  function _safeEqualHex(a, b){
    if (typeof a !== 'string' || typeof b !== 'string') return false;
    if (a.length !== b.length) return false;
    var buf1 = Buffer.from(a, 'hex');
    var buf2 = Buffer.from(b, 'hex');
    if (buf1.length !== buf2.length) return false;
    return crypto.timingSafeEqual(buf1, buf2);
  }

  app.get('/api/deliveries/:id', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    res.send({ id: id, slug: rec.slug, secret: rec.secret, created: rec.created });
  });

  app.post('/api/deliveries/:id/publish', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var payload = (req.body && req.body.payload) || 'ping';
    var sig = _hmacHex(rec.secret, payload);
    var ts = Date.now();
    var ok = _safeEqualHex(sig, sig);
    res.send({ id: id, slug: rec.slug, payload: payload, signature: sig, ts: ts, verified: ok });
  });

  app.get('/api/deliveries/:id/log', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var log = (rec.log || [].slice(-20);
    res.send({ id: id, count: log.length, log: log });
  });

  app.get('/api/deliveries/:id/stats', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var log = (rec.log || [];
    var total = log.length;
    var verified = 0;
    var i;
    for (i = 0; i < log.length; i++) { if (log[i].verified) verified++; }
    var failed = total - verified;
    var rate = (total > 0) ? (verified / total) : null;
    res.send({ id: id, total: total, verified: verified, failed: failed, rate: rate });
  });

  app.get('/api/deliveries/:id/latest', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var log = (rec.log || [];
    if (log.length === 0) return res.status(204).send('');
    var last = log[log.length - 1];
    res.send({ id: id, slug: rec.slug, last: last });
  });

  app.post('/api/deliveries/:id/rotate', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var old = rec.secret;
    rec.secret = _mk();
    rec.created = Date.now();
    res.send({ id: id, slug: rec.slug, old: old, secret: rec.secret, created: rec.created });
  });

  app.delete('/api/deliveries/:id', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var slug = rec.slug;
    delete _deliveries[id];
    res.send({ id: id, slug: slug, deleted: true });
  });

  app.get('/api/deliveries/:id/summary', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var log = (rec.log || [];
    var last = (log.length > 0) ? log[log.length - 1] : null;
    res.send({ id: id, slug: rec.slug, secret: rec.secret, created: rec.created, log_count: log.length, last: last });
  });

  app.get('/api/deliveries/:id/history', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var log = (rec.log || [];
    var out = [];
    var i;
    for (i = 0; i < log.length; i++) {
      var e = log[i];
      out.push({
        idx: i,
        phase: e.phase,
        ok: e.ok,
        detail: e.detail
      });
    }
    res.send({ id: id, slug: rec.slug, count: out.length, history: out });
  });

  app.get('/api/deliveries/:id/payload', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var payload = (req.query && req.query.payload) || 'ping';
    var sig = _hmacHex(rec.secret, payload);
    res.send({ id: id, slug: rec.slug, payload: payload, signature: sig, length: payload.length });
  });

  app.post('/api/deliveries/:id/verify', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var payload = (req.body && req.body.payload) || '';
    var sig = (req.body && req.body.signature) || '';
    var ok = _safeEqualHex(sig, _hmacHex(rec.secret, payload));
    res.send({ id: id, slug: rec.slug, ok: ok, signature: sig, payload: payload });
  });

  app.get('/api/deliveries/:id/signature', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var payload = (req.query && req.query.payload) || 'ping';
    var sig = _hmacHex(rec.secret, payload);
    res.send({ id: id, slug: rec.slug, payload: payload, signature: sig, length: payload.length });
  });

  app.get('/api/deliveries/:id/secret', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var reveal = (req.query && req.query.reveal);
    var secret = rec.secret;
    if (reveal !== '1') secret = secret.substr(0, 4) + '...';
    res.send({ id: id, slug: rec.slug, secret: secret, revealed: (reveal === '1') });
  });

  app.get('/api/deliveries/:id/created', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var now = Date.now();
    var age = now - rec.created;
    res.send({ id: id, slug: rec.slug, created: rec.created, now: now, age_ms: age, age_s: Math.floor(age / 1000) });
  });

  app.get('/api/deliveries/:id/age', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var now = Date.now();
    var age = now - rec.created;
    var days = Math.floor(age / 86400000);
    var hours = Math.floor(age / 3600000);
    var minutes = Math.floor(age / 60000);
    var seconds = Math.floor(age / 1000);
    res.send({ id: id, slug: rec.slug, created: rec.created, now: now, age_ms: age, age_s: seconds, age_m: minutes, age_h: hours, age_d: days });
  });

  app.get('/api/deliveries/:id/log/:idx', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    res.send({ id: id, slug: rec.slug, idx: idx, phase: e.phase, ok: e.ok, detail: e.detail });
  });

  app.get('/api/deliveries/:id/log/:idx/raw', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    var lines = [];
    lines.push('phase=' + (e.phase || ''));
    lines.push('ok=' + (e.ok ? '1' : '0'));
    lines.push('detail=' + (e.detail || ''));
    res.send(lines.join('\n'));
  });

  app.get('/api/deliveries/:id/log/:idx/pretty', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    var lines = [];
    lines.push('Phase: ' + (e.phase || '(none)'));
    lines.push('Ok: ' + (e.ok ? 'yes' : 'no'));
    lines.push('Detail: ' + (e.detail || '(none)'));
    res.send(lines.join('\n'));
  });

  app.get('/api/deliveries/:id/log/:idx/next', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length - 1) return res.status(404).send('no next log entry');
    var e = log[idx + 1];
    res.send({ id: id, slug: rec.slug, idx: idx + 1, phase: e.phase, ok: e.ok, detail: e.detail });
  });

  app.get('/api/deliveries/:id/log/:idx/prev', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 1 || idx >= log.length) return res.status(404).send('no prev log entry');
    var e = log[idx - 1];
    res.send({ id: id, slug: rec.slug, idx: idx - 1, phase: e.phase, ok: e.ok, detail: e.detail });
  });

  app.get('/api/deliveries/:id/log/:idx/count', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    res.send({ id: id, slug: rec.slug, idx: idx, phase: e.phase, ok: e.ok, detail: e.detail, total: log.length, remaining: log.length - idx - 1 });
  });

  app.get('/api/deliveries/:id/log/:idx/ts', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    res.send({ id: id, slug: rec.slug, idx: idx, phase: e.phase, ts: e.ts, iso: (e.ts ? new Date(e.ts).toISOString() : null) });
  });

  app.get('/api/deliveries/:id/log/:idx/age', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    var now = Date.now();
    var age = (e.ts ? now - e.ts : null);
    res.send({ id: id, slug: rec.slug, idx: idx, phase: e.phase, ts: e.ts, now: now, age_ms: age, age_s: (age ? Math.floor(age / 1000) : null) });
  });

  app.get('/api/deliveries/:id/log/:idx/diff', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 1 || idx >= log.length) return res.status(404).send('no prev log entry');
    var a = log[idx - 1];
    var b = log[idx];
    var diff = {};
    diff.phase = (a.phase === b.phase) ? null : { from: a.phase, to: b.phase };
    diff.ok = (a.ok === b.ok) ? null : { from: a.ok, to: b.ok };
    diff.detail = (a.detail === b.detail) ? null : { from: a.detail, to: b.detail };
    res.send({ id: id, slug: rec.slug, idx: idx, diff: diff });
  });

  app.get('/api/deliveries/:id/log/:idx/compare', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length - 1) return res.status(404).send('no next log entry');
    var a = log[idx];
    var b = log[idx + 1];
    var diff = {};
    diff.phase = (a.phase === b.phase) ? null : { from: a.phase, to: b.phase };
    diff.ok = (a.ok === b.ok) ? null : { from: a.ok, to: b.ok };
    diff.detail = (a.detail === b.detail) ? null : { from: a.detail, to: b.detail };
    res.send({ id: id, slug: rec.slug, idx: idx, diff: diff });
  });

  app.get('/api/deliveries/:id/log/:idx/ok', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    res.send(e.ok ? 'yes' : 'no');
  });

  app.get('/api/deliveries/:id/log/:idx/phase', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    res.send(e.phase || '');
  });

  app.get('/api/deliveries/:id/log/:idx/detail', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    res.send(e.detail || '');
  });

  app.get('/api/deliveries/:id/log/:idx/ts', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    res.send(e.ts ? new Date(e.ts).toISOString() : '');
  });

  app.get('/api/deliveries/:id/log/:idx/age', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    var now = Date.now();
    var age = (e.ts ? now - e.ts : null);
    res.send(age ? Math.floor(age / 1000) + 's' : '');
  });

  app.get('/api/deliveries/:id/log/:idx/diff', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 1 || idx >= log.length) return res.status(404).send('no prev log entry');
    var a = log[idx - 1];
    var b = log[idx];
    var diff = {};
    diff.phase = (a.phase === b.phase) ? null : { from: a.phase, to: b.phase };
    diff.ok = (a.ok === b.ok) ? null : { from: a.ok, to: b.ok };
    diff.detail = (a.detail === b.detail) ? null : { from: a.detail, to: b.detail };
    res.send({ id: id, slug: rec.slug, idx: idx, diff: diff });
  });

  app.get('/api/deliveries/:id/log/:idx/compare', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length - 1) return res.status(404).send('no next log entry');
    var a = log[idx];
    var b = log[idx + 1];
    var diff = {};
    diff.phase = (a.phase === b.phase) ? null : { from: a.phase, to: b.phase };
    diff.ok = (a.ok === b.ok) ? null : { from: a.ok, to: b.ok };
    diff.detail = (a.detail === b.detail) ? null : { from: a.detail, to: b.detail };
    res.send({ id: id, slug: rec.slug, idx: idx, diff: diff });
  });

  app.get('/api/deliveries/:id/log/:idx/ok', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    res.send(e.ok ? 'yes' : 'no');
  });

  app.get('/api/deliveries/:id/log/:idx/phase', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    res.send(e.phase || '');
  });

  app.get('/api/deliveries/:id/log/:idx/detail', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    res.send(e.detail || '');
  });

  app.get('/api/deliveries/:id/log/:idx/ts', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    res.send(e.ts ? new Date(e.ts).toISOString() : '');
  });

  app.get/api/deliveries/:id/log/:idx/age', (req, res) => {
    var id = req.params.id;
    var rec = _deliveries[id];
    if (!rec) return res.status(404).send('not found');
    var idx = parseInt(req.params.idx, 10);
    var log = (rec.log || [];
    if (isNaN(idx) || idx < 0 || idx >= log.length) return res.status(404).send('no such log entry');
    var e = log[idx];
    var now = Date.now();
    var age = (e.ts ? now - e.ts : null);
    res.send(age ? Math.floor(age / 1000) + 's' : '');
  });

  // Graceful shutdown
  process.on('SIGINT', function(){
    console.log('[SIGINT] shutdown requested, flushing queue...');
    process.exit(0);
  });
  process.on('SIGTERM', function(){
    console.log('[SIGTERM] shutdown requested, flushing queue...');
    process.exit(0);
  });
```